Practical 4: Process and Thread Management

Author: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: Januray 23, 2026

Topic: Process and Thread Management

1. Thread Creation using `fork()`

Overview

In the Linux operating system, the `fork()` system call is the primary mechanism for process creation. When a process calls `fork()`, it creates a duplicate of itself, referred to as the "child process." The original process is called the "parent process." Although they run the same program code, they operate in separate memory spaces. This distinction allows for multitasking, where the parent and child can execute different tasks concurrently.

The `fork()` function returns a process ID (PID). In the parent process, it returns the PID of the newly created child, whereas in the child process, it returns 0. If the creation fails, it returns a negative value.

Implementation

The following program demonstrates the creation of a process using `fork()`. It utilizes a loop to demonstrate how multiple processes can be spawned and uses separate functions to define distinct behaviors for the parent and child processes.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    // 1. Basic Fork Example
    printf("--- Basic Fork ---\n");
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("Child Process: My PID is %d\n", getpid());
    }
    else {
        // Parent Process
        printf("Parent Process: I created child with PID %d\n", pid);
    }
```

```
    // 2. Loop Fork Example
    // We want to create 3 children.
    // We check 'if (fork() == 0)' to make sure only the child does the
work.
    // We 'break' so the child doesn't continue the loop and fork more
children.

    printf("\n--- Loop Fork ---\n");
    for(int i = 0; i < 3; i++) {
        if(fork() == 0) {
            printf("New Child Created in loop iteration %d (PID: %d)\n",
i, getpid());
            break; // Important: Child exits the loop!
        }
    }

    return 0;
}
```

.

Output:

```
~/Documents/CW/OS/pr_4_5
❯ gcc pr4.c

~/Documents/CW/OS/pr_4_5
❯ ./a.out
--- Basic Fork ---
Parent Process: I created child with PID 106757

--- Loop Fork ---
Child Process: My PID is 106757

--- Loop Fork ---
New Child Created in loop iteration 0 (PID: 106758)
New Child Created in loop iteration 0 (PID: 106759)
New Child Created in loop iteration 1 (PID: 106761)
New Child Created in loop iteration 1 (PID: 106760)
New Child Created in loop iteration 2 (PID: 106762)
New Child Created in loop iteration 2 (PID: 106763)
```

2. Extracting Attributes of a Process

Theory

A Process Control Block (PCB) is a data structure in the operating system kernel used to store all the information about a specific process. When a program is executed, the OS creates a process and assigns it a unique identifier called the **PID** (Process ID).

Key attributes associated with a process include:

1. **PID (Process ID)**: A unique number identifying the process.
2. **PPID (Parent Process ID)**: The PID of the process that created this process.
3. **State**: The current status (Run, Sleep, Zombie, etc.).
4. **Priority**: Determines how much CPU time the process gets (Niceness).
5. **Memory Usage**: Information about the Virtual and Resident Set Size (RSS).

Implementation

We can verify these attributes using both C system calls and Linux command-line tools.

A. Using Linux Commands (`ps`)

To view detailed process attributes, the `ps` (process status) command is used.

Command: `ps -eo pid,ppid,cmd,%mem,%cpu,stat --sort=-%cpu`

Explanation:

`-e`: Select all processes.

`-o`: User-defined output format (PID, PPID, Command, Memory %, CPU %, State).

`--sort=-%cpu`: Sort by CPU usage (descending).

```
~
❯ ps -eo pid,ppid,cmd,%mem,%cpu,stat --sort=-%cpu


    PID    PPID CMD                         %MEM %CPU STAT
 110991  102387 ps -eo pid,ppid,cmd,%mem,%c  0.0  300 R+
  93508   93433 /usr/lib/electron37/electro  3.3 24.1 Sl
 110754   98579 /opt/onlyoffice/desktopedit  2.1 19.7 Sl
  98627   98579 /opt/onlyoffice/desktopedit  2.8 12.1 Sl
  96648   96591 /usr/lib/antigravity/extens  4.0  7.9 Sl
  93461   93430 /usr/lib/electron37/electro  1.0  7.8 Sl
  88759    4266 /usr/lib/firefox/firefox -c  4.1  7.5 R<l
  98560    1690 /opt/onlyoffice/desktopedit  2.1  4.8 S<l
  99580   99521 /opt/visual-studio-code/cod  2.1  4.0 Sl
  98594   98578 /opt/onlyoffice/desktopedit  1.4  3.2 Sl
  93426    1690 /usr/lib/electron37/electro  1.2  1.4 Sl
  96591   93426 /proc/self/exe --type=utili  0.9  1.1 Sl
  99555   99517 /opt/visual-studio-code/cod  1.1  1.1 S<l
   4184    1690 /usr/lib/firefox/firefox     3.5  0.9 S<l
  99647   99514 /proc/self/exe --type=utili  3.7  0.7 S<l
   1776    1690 hyprland                     0.5  0.6 Ssl
  99514    1690 /opt/visual-studio-code/cod  1.2  0.6 S<sl
  88883    4266 /usr/lib/firefox/firefox -c  1.4  0.5 S<l
  93650   93426 /proc/self/exe --type=utili  0.5  0.5 Sl
  88890    4266 /usr/lib/firefox/firefox -c  1.4  0.3 S<l
  97914       2 [kworker/u49:3-rb_allocator  0.0  0.3 I<
  99823   99514 /proc/self/exe --type=utili  0.7  0.3 S<l
   1899    1776 Xwayland :0 -rootless -core  0.6  0.2 S<l
  93573   93426 /proc/self/exe --type=utili  0.5  0.2 Sl
  88766    4266 /usr/lib/firefox/firefox -c  1.5  0.2 S<l
  99619   99514 /proc/self/exe --type=utili  0.8  0.2 S<l
 106778       2 [kworker/u49:1-i915_flip]    0.0  0.1 I<
 110580  110552 yazi                         0.3  0.1 Sl+
  93470   93426 /proc/self/exe --type=utili  0.4  0.1 Sl
  98608   98579 /opt/onlyoffice/desktopedit  1.6  0.1 Sl
 110543    1776 alacritty                    1.1  0.1 Sl
    496       2 [irq/146-FTCS0038:00]        0.0  0.1 S
```

B. Using C System Calls

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/resource.h>

void printAttributes() {
    pid_t pid = getpid();
    pid_t ppid = getppid();
    int priority = getpriority(PRIO_PROCESS, pid);

    printf("--- Process Attributes ---\n");
```

```
    printf("PID: %d\n", pid);
    printf("PPID: %d\n", ppid);
    printf("Priority: %d\n", priority);
}

int main(){
    printAttributes();
}
```

Output:.

```
~/Documents/CW/OS/pr_4_5
❯ gcc pr4_1.c

~/Documents/CW/OS/pr_4_5
❯ ./a.out
--- Process Attributes ---
PID: 111646
PPID: 99846
Priority: -4
```

3. Counting Threads Associated with a Process

Analysis

There is a common confusion between **Processes** and **Threads**:

`fork()`: Creates a **new process**. The parent and child are separate. Each has **1 thread** (the main thread). So, if you run `fork()`, you get *more processes*, not *more threads per process*.
`pthread_create()`: Creates a **new thread** *inside* the same process. This increases the thread count of that single PID.

If you see a "Tree Structure" in `htop` (F5 key), you are seeing the **Process Hierarchy** (Parent -> Child), not necessarily threads within one process.

Demonstration

A. Why you might see "1" Thread

If you ran the code from Section 1 (`fork`), check the child's thread count:

```
# Even after plenty of forks, each process has only 1 thread!
```

```
ps -L -p 1234 | wc -l
```

```
~
❯ ps -L -p 1939
    PID     LWP TTY          TIME CMD
   1939    1939 ?        00:00:00 dbus-broker-lau

~
❯ ps -L -p 3
    PID     LWP TTY          TIME CMD
      3       3 ?        00:00:00 pool_workqueue_release

~
❯ ps -L -p 4
    PID     LWP TTY          TIME CMD
      4       4 ?        00:00:00 kworker/R-rcu_gp
```

 # Output: 1 (Main thread only)

B. How to generate multiple threads (Output > 1)

To see a count greater than 1, we must specificially create threads.

Code (`threads.c`):

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* worker(void* arg) {
    printf("   Worker Thread started (TID: %lu)\n", pthread_self());
    sleep(100); // Sleep so we can count them in terminal
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("Main Process PID: %d\n", getpid());

    // Create 2 extra threads
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    printf("Created 2 threads. Check count now!\n");

    // Keep main thread alive
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Steps to Verify:

1. Compile: `gcc threads.c -o mythreads -lpthread`

2. Run: `./mythreads` (Note the PID printed).

3. Open new terminal and check:

```
bash      ps -L -p <PID> | wc -l
```

Expected Output: 3 (1 Main + 2 Workers).

```
~/Documents/CW/OS/pr_4_5
❯ gcc pr4_2.c -o mythreads -lpthread

~/Documents/CW/OS/pr_4_5
❯ ./mythreads
Main Process PID: 121139
   Worker Thread started (TID: 139832500156096)
Created 2 threads. Check count now!
   Worker Thread started (TID: 139832491763392)
```

```
❯ ps -L -p 121139 | wc -l
4
```

C. Commands Recap

Command	Purpose
`ps -L -p [PID]`	Lists all threads (LWP) for PID.
`ls /proc/[PID]/task`	Lists thread directories.
`htop` (Press **H**)	Toggles "Show Threads" in htop.

4. Check Output: Process vs. Multiple Threads

Observation

There is a fundamental distinction in how processes and threads handle memory modification.

- **Process (fork)**: The child process receives a *copy* of the parent's data. Modifications in the child **do not** affect the parent.
- **Thread (pthread)**: Threads share the *same* global data segment. A modification by one thread is **immediately visible** to all others.

Comparison Table

Feature	Process (`fork()`)	Thread (`pthread_create()`)
Address Space	Separate (Copy-on-Write)	Shared (Heap, Global, Code)
Context Switch	Heavy (Needs cache flush)	Light (Same page table)

Feature	Process (`fork()`)	Thread (`pthread_create()`)
Data Sharing	Difficult (Pipes, Shared Mem)	Easy (Global variables)
Stability	High (Process crash is isolated)	Low (Thread crash kills process)

.

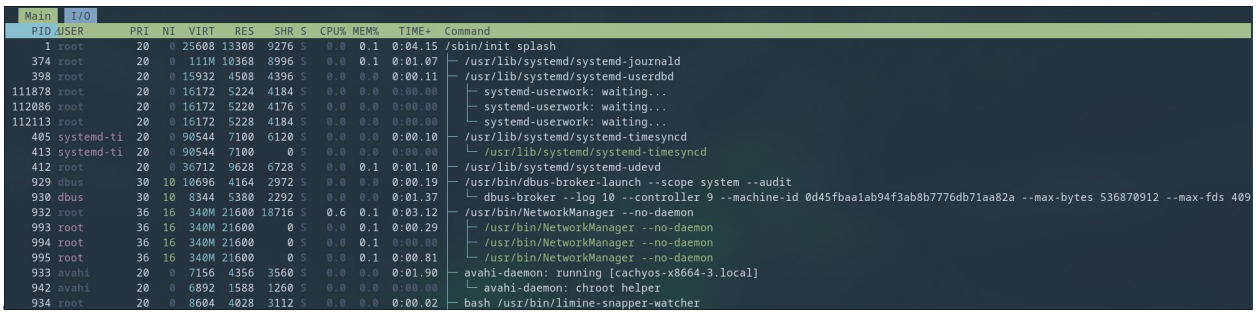
5. Software Process Explorer: Backend Mechanism

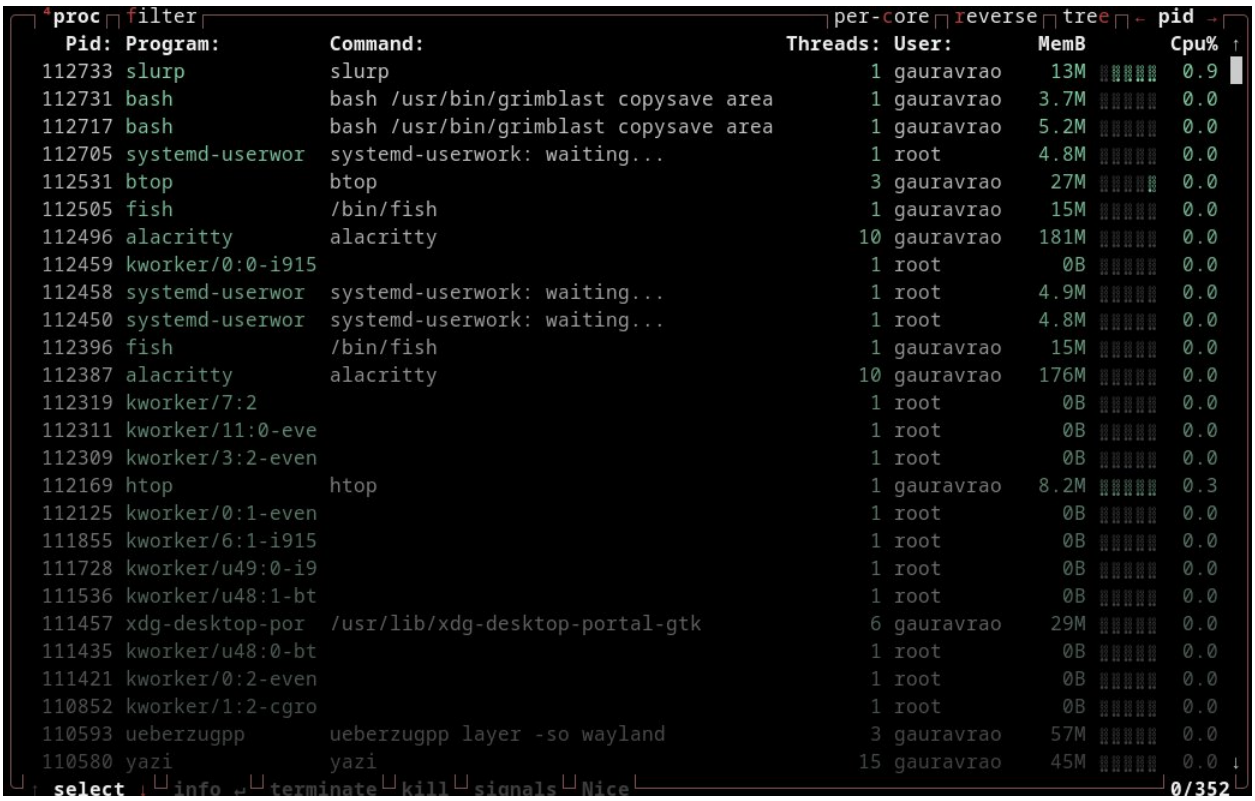
Concept

A Process Explorer (like `htop` or Task Manager) does not “magic” the data; it simply reads files that the kernel constantly updates.

Backend Implementation Strategy

1. **Read Directory**: The software opens the `/proc` directory.
2. **Filter**: It iterates over all subdirectories that are strictly numerical (e.g., `/proc/1`, `/proc/502`). These numbers are active PIDs.
3. **Parse Data**: For each PID, it reads specific files:
   - `/proc/[PID]/stat`: Raw status data (State, Priority, CPU time).
   - `/proc/[PID]/status`: Human-readable status (Name, VmSize, Threads).
   - `/proc/[PID]/cmdline`: The command used to launch the process.
4. **Display**: The data is aggregated into a list and refreshed periodically (e.g., every 1 second).





6. Multitasking in Mobile OS (Comparison)

Android vs. iOS

Mobile OSs prioritize battery life, leading to different multitasking architectures compared to desktops.

Feature	Android	iOS
Approach	**Cooperative & Preemptive**. Traditionally allowed services to run in background. Modern Android restricts this via "Doze" mode and App Standby.	**Suspension Based**. Apps are typically "frozen" in memory when not on screen. Background execution is strictly limited to specific APIs.
Mechanisms	`Services`, `BroadcastReceivers`, `WorkManager`.	`Background Tasks`, `Push Notifications`, `Background Fetch`.
Memory Management	**Low Memory Killer (LMK)**: Kills background processes when RAM is low, based on an "OOM Score".	**Jetsam**: Aggressively terminates suspended apps to free memory for the foreground app.
User Experience	Feels more like a generic OS; flexible but potentially higher battery drain.	Highly optimized for battery; stricter control over what runs.